

03 — Featurization: filtrations and persistence diagrams

17 September 2026

1 Theory

Aim

- Every pattern of 02 $\rightarrow$ one persistence diagram per filtration, H_0 and H_1
- Diagrams only: no vectorisation here, since every image/curve calibration is fitted per split (leakage otherwise)
- Three filtration types, one sweep parameter (k for DTM); each is a different clock on the same points

Filtrations

A filtration is a nested family of complexes indexed by a scale t ; persistent homology records the t at which components (H_0) and loops (H_1) are born and die.

- **Rips**: flag complex of the edges $d(x, y) \leq t$. Pure geometry; H_0 births all 0, so H_0 is the single-linkage merge heights (the Euclidean MST). H_1 = gaps between clusters
- **Alpha**: the Delaunay subcomplex at scale t , filtration value = circumradius of the smallest empty ball. Nerve of the union of balls $\Rightarrow$ homotopy equivalent to Čech, i.e. the exact topology of the t -neighbourhood, not the Rips over-approximation. $O(n)$ simplices in 2D
- **DTM_k** (Anai et al. 2020): weighted Rips with vertex weight

$$f(x) = \left(\frac{1}{k} \sum_{y \in N_k(x)} d(x, y)^q \right)^{1/q}, \quad q = 2,$$

$N_k(x)$ the k nearest neighbours of x (x itself included). Vertex enters at $2f(x)$, edge xy at $\max\{d(x, y) + f(x) + f(y), 2f(x), 2f(y)\}$

- Dense points enter early, sparse points late $\Rightarrow$ the diagram sees *local density variation*, not just geometry
- k is the density-estimation scale: small k resolves fine clustering but is noisy, large k smooths. Swept, like the radius r of an L function
- $k = 1$ gives $f \equiv 0$ and reduces to Rips (verified)

Why these three

- Rips: geometric baseline, the representation every published PH number in this project used
- Alpha: exact union-of-balls topology and it records empty-ball radii, i.e. spacing. Aimed at Matérn II, whose departure is a hard core (a forbidden spacing), the regime where the L test lags (02)
- DTM: the clustered families differ by *density* contrast; $k \in \{5, 10, 15, 20\}$ covers the density scales

Two implementation identities

- **DTM through Rips.** The DTM complex is a flag filtration, so it is determined by its vertex and edge values. Feeding ripser the matrix $M_{ij} = \max\{d_{ij} + f_i + f_j, 2f_i, 2f_j\}$, $M_{ii} = 2f_i$, reproduces GUDHI's diagrams exactly, in $O(n^2)$ memory instead of GUDHI's $O(n^3)$ 2-skeleton, $\approx 50\times$ faster. This removes the entire memory-management layer the old pipeline needed (chunk scheduling, per-worker RSS models, OOM retries)
- **Alpha scale.** GUDHI returns squared circumradii α . Stored as $2\sqrt{\alpha}$, so alpha and Rips share one axis (ball diameter = edge length). A monotone axis map does not change the diagram's topology, only the grid a later vectorisation puts on it; `scale` in the config switches between α , $\sqrt{\alpha}$, $2\sqrt{\alpha}$

Conventions

- $\max \dim = 1$: H_0, H_1
- Only finite pairs kept; the one essential H_0 bar (death = ∞) is dropped, so $|H_0| = n - 1$ always
- Zero-persistence pairs are not produced by either backend
- No thresholding: the filtration runs to completion

2 Implemented pipeline

Code

<code>src/cloudforger/featurization/filtrations.py</code>	<code>rips, alpha, dtm; tags</code>
<code>src/cloudforger/featurization/sweep.py</code>	<code>config, shard, merge</code>
<code>scripts/featurize.py</code>	CLI: <code>run, merge</code>
<code>slurm/featurize.sh</code>	one 48-CPU job: <code>run then merge</code>
<code>configs/featurization/config.yaml</code>	filtration list, max dim, shard size

Backends: ripser (Rips, DTM), GUDHI (alpha), scipy (kNN, distances). 235 lines in total.

Configuration

- `rips; alpha (scale: diameter); dtm k = 5, 10, 15, 20 (q = 2)  $\Rightarrow$  6 diagram sets  $\times$  5 families`
- `max dim = 1, shards of 500 patterns (40 per family)`
- One config entry = one output directory; adding a k is one line

Steps

1. **Run** (`run --jobs 48`): one file per (family, filtration, shard), atomic, resumable; a finished shard or a merged output is skipped
2. **Merge**: shards $\rightarrow$ `data/featurization/<family>/<tag>/diagrams.npz`, then the shards are deleted. Refuses to write (keeping the shards) on a missing or out-of-order shard, or a diagram count $\neq$ the manifest's

Output

`data/featurization/<family>/<tag>/diagrams.npz`, tags `rips, alpha_diameter, dtm_k5, ...`

- `case_id (P,)`, row order of 02's `manifest.csv` — the only join key back to θ
- `h0, h1`: all patterns' finite (b, d) pairs concatenated; `h<d>.offsets (P + 1,)` cuts pattern p out of them
- `spec`: the config entry that produced the file
- Same flat layout as 02's `points.npz`: one array, one offset vector, no pickles

Cost

Mean seconds per pattern (40 random patterns per family, $n \leq 961$):

	Poisson	Thomas	Nested	Matérn II	LGCP
Rips	0.015	0.027	0.027	0.018	0.045
Alpha	0.001	0.002	0.002	0.002	0.002
DTM ₂₀	0.025	0.043	0.042	0.031	0.075

- ≈ 5.6 CPU-h for all 30 (family, filtration) pairs $\Rightarrow \approx 15$ min on one 48-CPU node; ≈ 4.2 GB of diagrams
- Memory per worker is one distance matrix, $n^2 \cdot 8\text{B} \leq 8\text{MB}$: no scheduling needed, unlike the old GUDHI pipeline

Results

Check	Result	Reference
DTM vs GUDHI DTM <code>RipsComplex</code> , $k = 1, 5, 15$	identical (H_0, H_1)	exact
DTM _{$k=1$} vs Rips	identical	$f \equiv 0$
Alpha <code>diameter</code> vs $2\sqrt{\text{radius}}$	identical	monotone map
Merged slice vs pattern recomputed alone	identical (all 6 tags)	—
$ H_0 $ per pattern	$n - 1$	one essential bar dropped
Merge with a shard missing	refuses, shards kept	—

Mean pairs per point (20 patterns per family):

$ H_1 /n$	Poisson	Thomas	Nested	Matérn II	LGCP
Rips	0.24	0.20	0.21	0.24	0.22
Alpha	0.90	0.83	0.84	0.91	0.84
DTM ₅	0.19	0.15	0.16	0.18	0.16
DTM ₂₀	0.19	0.15	0.16	0.19	0.16

3 Notes

Interpretation

- Alpha carries $\approx 4\times$ the H_1 pairs of Rips: it keeps every Delaunay cycle, most of them short-lived. Information, but also a heavier tail for any vectorisation to absorb
- $|H_1|/n$ is nearly flat across families and k : the families differ in *where* the pairs sit, not how many there are — a total-count feature would be uninformative
- DTM's axis is in distance units but rescaled by local density, so its diagrams are not directly comparable with Rips/alpha; each filtration needs its own calibration downstream
- Clustered families cost more per pattern than Poisson at equal n (denser neighbourhoods $\Rightarrow$ more simplices); LGCP is the most expensive

Scope and limits

- H_2 and higher are empty in 2D; max dim = 1 is not a restriction
- The essential H_0 bar is constant across patterns and carries nothing; n itself is in the manifest
- Diagrams are the full sufficient output of this stage: vectorisation (images, landscapes, Betti curves) is fitted per training split later, never here

- No edge correction: the filtration sees W as it is, unlike $\hat{K}$ in 01–02. Boundary effects are part of the representation and identical across families

Open

- Which k (or which subset) carries the signal; the sweep exists to answer it
- Whether alpha beats Rips on Matérn II, the reason it is here
- Alpha axis choice $(\alpha, \sqrt{\alpha}, 2\sqrt{\alpha})$ is irrelevant to the topology but not to a fixed vectorisation grid
- Whether short bars should be pruned before vectorisation (alpha especially)